The Journal of Open Source Software

sklearn-migrator: Safely Migrating scikit-learn Models Across Versions

Alberto Valdés [1]

1 Independent Researcher

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 11 August 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

`sklearn-migrator` is a Python library for **safely migrating scikit-learn models across versions** while preserving inference behavior and remaining robust to attribute changes. Given scikit-learn's broad adoption in data science and industry, we focus on eight high-usage estimators (trees, ensembles, and linear/logistic models) that frequently appear in practitioner surveys and usage reports (*Kaggle's State of Machine Learning and Data Science 2021*, n.d.; Pedregosa et al., 2011).

The migration proceeds in two stages. **Stage 1 (parity):** the library captures a minimal set of prediction-critical attributes ("constructor parameters") that guarantee parity of outputs across versions; these are used to reconstruct an equivalent estimator in the target version and validated under a strict tolerance (e.g., `max |y_in - y_out| < 1e-4`). **Stage 2 (compatibility):** remaining attributes are serialized with a version-aware policy that gracefully handles additions, removals, and renames so deserialization does not break across releases. This approach directly addresses the well-known **version fragility** of pickle/joblib persistence with scikit-learn models as documented in the official guidelines and prior work (Fitzpatrick & Manning, 2024; *Model Persistence*, n.d.; Parida et al., 2025).

Concretely, consider version `0.21.3` exposing `param_1, param_2, param_3, param_4` and version `1.7.0` exposing `param_1, param_2, param_3, param_5`. We use `param_1` and `param_2` to enforce identical predictions across versions. Since `param_3` exists everywhere, it is stored in the payload and directly reassigned on load. For version-specific attributes, the serializer records values for both `param_4` and `param_5`: in `0.21.3`, the payload stores the real value of `param_4` and the **default** value that `param_5` would take in newer versions; in `1.7.0`, it stores the real value of `param_5` and the default value that `param_4` would take in older versions. During deserialization, attributes are assigned using a simple `try/except` (or `hasattr`) so only valid attributes for the target version are set, and missing ones are safely skipped.

This submission covers eight widely used estimators across classification and regression—`DecisionTreeClassifier`, `RandomForestClassifier`, `GradientBoostingClassifier`, `LogisticRegression`, `DecisionTreeRegressor`, `RandomForestRegressor`, `GradientBoostingRegressor` and `LinearRegression`—and validates representative cross-version pairs from `0.21.x` through `1.7.x` by asserting prediction parity on fixed synthetic datasets. Continuous integration on multiple Python versions, unit tests, and an MIT license support reproducibility and adoption in production MLOps workflows.

Statement of need

Persisted scikit-learn models frequently break across library upgrades because internal attributes, defaults, and serialization details change over time. Standard persistence mechanisms (e.g., pickle/joblib) are **version-fragile**: a model saved under one release may fail to load—or load with

altered behavior—under another. This is cautioned in the official documentation and has been observed in empirical and experiential studies (Fitzpatrick & Manning, 2024; *Model Persistence*, n.d.; Parida et al., 2025). The result complicates production upgrades, environment migrations, and cross-team sharing, and undermines the long-term reproducibility required in MLOps, audits, and regulated workflows.

Given scikit-learn's broad adoption in research and industry, there is practical value in keeping legacy models usable across releases (*Kaggle's State of Machine Learning and Data Science 2021*, n.d.; Pedregosa et al., 2011). The estimators covered in this submission—DecisionTree/RandomForest/GradientBoosting for classification and regression, plus Logistic/Linear Regression—are among the most commonly taught and deployed models in applied machine learning, frequently appearing in practitioner surveys and reports (*Kaggle's State of Machine Learning and Data Science 2021*, n.d.).

`sklearn-migrator` addresses this need with a two-stage, version-aware (de)serialization strategy. First, it captures the minimal, prediction-critical attributes to guarantee parity of outputs between versions. Second, it serializes remaining attributes with explicit, version-conditioned defaults so that parameters added, removed, or renamed across releases do not break deserialization. Unlike pickle/joblib, the library uses portable, JSON-compatible Python dictionaries, enabling safe transport, inspection, and storage independent of the original runtime.

The library targets practitioners and MLOps teams who must migrate or reproduce models across heterogeneous environments. It supports forward and backward migration and has been exercised across 30 scikit-learn releases (`0.21.x` → `1.7.x`), covering **900** version pairs with unit tests and environment-isolated validation. This foundation reduces upgrade risk today while remaining extensible to additional estimators and components in future releases.

Example

The example below trains a `RandomForestRegressor`, serializes it to a portable (JSON-compatible) dictionary, deserializes it, and checks prediction parity within a strict tolerance. In practice, the `version_out` may correspond to a different scikit-learn installation (e.g., upgrading from 1.5.x to 1.7.x).

```python
import json
import numpy as np
import sklearn
from sklearn.datasets import make_regression
from sklearn.ensemble import RandomForestRegressor

from sklearn_migrator.regression.random_forest_reg import (
    serialize_random_forest_reg,
    deserialize_random_forest_reg,
)

# Train in the "source" environment
X, y = make_regression(n_samples=200, n_features=10, random_state=0)
src_version = sklearn.__version__

src_model = RandomForestRegressor(
    n_estimators=50, random_state=0
).fit(X, y)

# Serialize to a portable payload (JSON-compatible dict)
payload = serialize_random_forest_reg(src_model, version_in=src_version)
```

DRAFT

```python
# (Optional) store as JSON
with open("model.json", "w") as f:
    json.dump(payload, f)

# --- In a different environment, load and deserialize ---
# (For illustration we reuse the same environment; in practice, version_out may differ.)
tgt_version = sklearn.__version__
with open("model.json") as f:
  payload_loaded = json.load(f)

tgt_model = deserialize_random_forest_reg(payload_loaded, version_out=tgt_version)

# Prediction parity check
y_src = src_model.predict(X)
y_tgt = tgt_model.predict(X)
assert np.max(np.abs(y_src - y_tgt)) < 1e-4
print("Prediction parity verified.")
```

Analogous functions exist for all covered estimators:

- **Classification:** `DecisionTreeClassifier`, `RandomForestClassifier`, `GradientBoostingClassifie`
  `LogisticRegression`

- **Regression:** `DecisionTreeRegressor`, `RandomForestRegressor`, `GradientBoostingRegressor`,
  `LinearRegression`

The payload uses only JSON-encodable types, making it easy to store, transport, and inspect across environments. In a true two-environment workflow, the serialization code runs in the source environment (e.g., 1.5.0) and the deserialization code runs in the target environment (e.g., 1.7.0); the assertion above remains the same.

Limitations

- **Two environments required.** End-to-end validation relies on two isolated environments: a *source* environment (`version_in`) to serialize and a *target* environment (`version_out`) to deserialize and verify prediction parity. While this can be simulated on one machine, truly isolated setups (e.g., Docker images) are recommended to avoid dependency leakage.
- **Incomplete model coverage (work in progress).** This submission covers eight widely used estimators. Additional models (e.g., SVC/SVR, KNN, regularized linear models, and pipelines/components) are not yet supported but are planned for future releases. We actively welcome community contributions to expand coverage.

Acknowledgements

We thank the scikit-learn core developers and contributors for their open-source work and documentation, as well as the broader NumPy/SciPy/joblib ecosystems on which this project depends. We are grateful to colleagues and early adopters for testing and feedback that shaped the design, and to the JOSS editors and reviewers for guidance during the review process.

Fitzpatrick, P. C., & Manning, J. R. (2024). Davos: A python package "smuggler" for constructing lightweight reproducible notebooks. *SoftwareX*, *25*, 101614. https://doi.org/10.1016/j.softx.2023.101614

Kaggle's state of machine learning and data science 2021. (n.d.). https://storage.googleapis.com/kaggle-media/surveys/Kaggle%27s%20State%20of%20Machine%20Learning%20and%20Data%20Science%202021.pdf.

DRAFT

Model persistence. (n.d.). https://scikit-learn.org/stable/model_persistence.html.

Parida, S. K., Gerostathopoulos, I., & Bogner, J. (2025). *How do model export formats impact the development of ML-enabled systems? A case study on model integration*. https://doi.org/10.48550/arXiv.2502.00429

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, *12*, 2825–2830.

DRAFT